

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN &
TRUYỀN THÔNG VIỆT HÀN

Khoa Kỹ Thuật Máy Tính & Điện Tử



Triển khai bản thiết kế VLSI cho thuật toán mã hóa và giải mã ASCON

Sinh viên thực hiện: **Lý Hữu Lộc**

Đặng Anh Cường

Lớp:

21CE1

MỤC LỤC

DANH MỤC HÌNH ẢNH	iii
DANH MỤC BẢNG BIỂU	iv
MỞ ĐẦU.....	v
CHƯƠNG 1. CƠ SỞ LÝ THUYẾT	1
1.1 Bối cảnh và động cơ nghiên cứu	1
1.2 Tổng quan về thuật toán ASCON.....	1
1.2.1. Cấu trúc của ASCON	1
1.2.2. Quá trình hoạt động của ASCON.....	2
1.2.3. Tính bảo mật và hiệu quả	3
1.3 Mục tiêu và phạm vi báo cáo	3
1.4. Các công cụ thiết kế và phát triển.....	4
1.4.1. Ngôn ngữ mô tả phần cứng Verilog.....	4
1.4.2. Công cụ tổng hợp logic	5
1.4.2 OpenROAD.....	5
1.4.3. Công cụ kiểm tra DRC và LVS.....	6
1.4.4. Công cụ xem và chỉnh sửa layout	6
Kết Chương 1	7
CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC RTL.....	8
2.1 Thiết kế các khối chức năng	8
2.1.1. Thông số kỹ thuật trong ASCON.....	8
2.1.2. Module chính ASCON.....	8
2.1.3. Các module phụ trong ASCON.....	13
2.2 . Thiết kế RTL cho ASCON	15
2.2.1 Sơ đồ logic.....	15
Kết chương 2	16
CHƯƠNG 3. HOÀN THIỆN LAYOUT VÀ KẾT QUẢ.....	17
3.1 Quá trình Floorplaning	17
3.2 Quá trình Place và Route	17
3.3 Quá trình hoàn thiện layout	19
3.4 Đặc tả của thiết kế và phân tích.....	20
Kết Chương 3	20
KẾT LUẬN	21
TÀI LIỆU THAM KHẢO.....	viii

DANH MỤC HÌNH ẢNH

Hình 1. 1: Quá trình mã hoá sử dụng thuật toán ASCON.....	2
Hình 1. 2: Quá trình giải mã sử dụng thuật toán ASCON.....	2
Hình 1. 3: Flow về khung thiết kế mã nguồn mở OpenLane	4
Hình 1. 4: Ngôn ngữ mô tả phần cứng Verilog	4
Hình 1. 5: Logo của công cụ tổng hợp Yosys	5
Hình 1. 6: Hình ảnh các công cụ trong OpeaROAD thực hiện ra layout.....	5
Hình 1. 7: Logo của công cụ Magic	6
Hình 1. 8: Logo của công cụ Klayout.....	6
Hình 2. 1: Các giai đoạn hoạt động của ASCON	8
Hình 2. 2: Bước khởi tạo	10
Hình 2. 3: Thuật toán mã hoá và giải mã của ASCON	12
Hình 2. 4: Đăng ký các từ của trạng thái 320 bit S và hoạt động của module Permutation.....	13
Hình 2. 5: Kiến trúc module Substitution Layer	14
Hình 2. 6: Chức năng của module Linear Diffusion Layer.....	15
Hình 2. 7: Block diagram của ASCON mã hoá	15
Hình 2. 8: Block diagram của ASCON giải mã	15
Hình 2. 9: Quá trình xác minh đúng về chức năng của ASCON	16
Hình 2. 10: Wareform của ASCON	16
Hình 3. 1: Quá trình tạo khung để chứa các Strandard Cell.....	17
Hình 3. 2: Quá trình đặt các cell vào bên trong khung.....	18
Hình 3. 3: Quá định định tuyến của các cell với nhau	18
Hình 3. 4: Quá trình hoàn thiện layout của ASCON.....	19

DANH MỤC BẢNG BIỂU

Bảng 2. 1: Bảng thông số kỹ thuật trong ASCON	8
Bảng 2. 2: $S = x_0 x_1 x_2 x_3 x_4$	9
Bảng 2. 3: Bảng round constraint c_r được sử dụng mỗi vòng	14
Bảng 2. 4: Bảng tra cứu hộp S 5 bit $S(x)$	14
Bảng 3. 1: Bảng tổng hợp thông số cần thiết kế layout hoàn thành	20

MỞ ĐẦU

ASCON là một thuật toán mã hóa nhẹ (lightweight cryptography) được thiết kế để đáp ứng nhu cầu bảo mật trong các hệ thống IoT, thiết bị nhúng, và các ứng dụng yêu cầu hiệu năng cao nhưng hạn chế tài nguyên. Thuật toán này nổi bật với khả năng bảo mật mạnh mẽ, hiệu suất xử lý nhanh và tính linh hoạt trong triển khai. Đặc biệt, ASCON đã được chuẩn hóa bởi NIST như một thuật toán mật mã nhẹ lý tưởng cho các ứng dụng thực tế.

Việc triển khai thuật toán ASCON trên nền tảng VLSI có ý nghĩa quan trọng, giúp tối ưu hóa hiệu năng, giảm tiêu thụ năng lượng và đảm bảo độ tin cậy cao trong các ứng dụng phần cứng chuyên dụng.

Mục tiêu của nghiên cứu này là thiết kế toàn bộ quy trình từ mức RTL đến GDSII, đảm bảo thuật toán ASCON hoạt động hiệu quả trên phần cứng thực tế, đáp ứng các yêu cầu về tốc độ, diện tích và năng lượng tiêu thụ.

.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1 Bối cảnh và động cơ nghiên cứu

Trong bối cảnh phát triển nhanh chóng của các thiết bị Internet vạn vật (IoT) và các hệ thống nhúng, nhu cầu về các thuật toán mật mã nhẹ (lightweight cryptography) ngày càng tăng. Các ứng dụng này đòi hỏi các giải pháp bảo mật có khả năng hoạt động hiệu quả trên phần cứng tài nguyên hạn chế, đồng thời đảm bảo mức độ bảo mật cao. Đáp ứng yêu cầu này, ASCON đã được chọn làm tiêu chuẩn trong cuộc thi NIST nhờ sự kết hợp giữa tính đơn giản, hiệu quả và khả năng bảo vệ mạnh mẽ trước các tấn công hiện đại.

Tuy nhiên, việc hiện thực hóa các thuật toán mật mã như ASCON trên phần cứng là một thách thức kỹ thuật quan trọng. Thiết kế VLSI của một thuật toán mật mã không chỉ đòi hỏi sự hiểu biết sâu sắc về thuật toán mà còn liên quan đến việc chuyển đổi các thao tác lý thuyết thành các mạch logic khả thi.

Động cơ của nghiên cứu này là tạo ra một thiết kế VLSI hoàn chỉnh cho ASCON, nhằm mục đích kiểm chứng chức năng và cung cấp nền tảng để tích hợp vào các hệ thống bảo mật phần cứng. Thiết kế này không chỉ phục vụ cho mục tiêu nghiên cứu mà còn góp phần làm phong phú thêm tài nguyên thực hành cho các ứng dụng mật mã nhẹ trong tương lai.

1.2 Tổng quan về thuật toán ASCON

ASCON là một thuật toán mật mã nhẹ (lightweight cryptography) được thiết kế nhằm đáp ứng các yêu cầu bảo mật trên các hệ thống có tài nguyên hạn chế như IoT, thiết bị nhúng, và hệ thống cảm biến. ASCON đã được chọn làm tiêu chuẩn của NIST trong danh mục mật mã nhẹ nhờ các đặc điểm nổi bật về hiệu quả và bảo mật.

1.2.1. Cấu trúc của ASCON

ASCON là một thuật toán mã hóa dựa trên phương pháp sponge construction, kết hợp giữa các quá trình đệm (absorb) và trích xuất (squeeze).

Cấu trúc này được sử dụng để thực hiện hai chức năng chính:

- Mã hóa được xác thực (Authenticated Encryption - AE): Kết hợp mã hóa dữ liệu và tạo mã xác thực để đảm bảo tính toàn vẹn.
- Hàm băm được xác thực (Authenticated Hash Function - Hash): Đảm bảo dữ liệu không bị thay đổi và xác minh tính xác thực của nguồn gửi.

ASCON, mặc dù hỗ trợ cả mã hóa được xác thực (Authenticated Encryption - AE) và hàm băm được xác thực (Authenticated Hash Function - Hash), nhưng trong thiết kế này, chỉ tập trung vào việc triển khai chức năng mã hóa và giải mã.

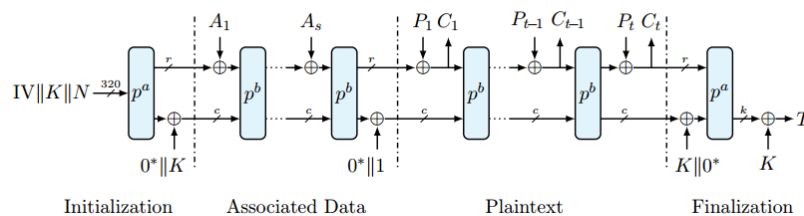
ASCON bao gồm các tham số chính như:

- Key (Khóa): Dùng để mã hóa và giải mã, với độ dài tiêu chuẩn là 128-bit.
- Nonce (Số ngẫu nhiên): Một giá trị không lặp lại trong mỗi phiên giao dịch, đảm bảo tính độc lập của từng bản mã.
- Associated Data (Dữ liệu liên kết): Được xác thực cùng với dữ liệu mã hóa, nhưng không được mã hóa.
- Plaintext (Dữ liệu gốc): Dữ liệu cần mã hóa hoặc giải mã.

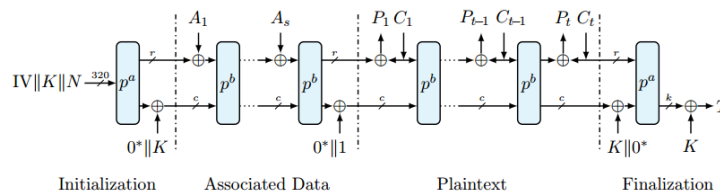
1.2.2. Quá trình hoạt động của ASCON

ASCON bao gồm ba giai đoạn chính:

1. Khởi tạo (Initialization): Trộn khóa và nonce thông qua các phép biến đổi vòng lặp (round transformations), thiết lập trạng thái ban đầu.
2. Xử lý dữ liệu liên kết (Associated Data Processing): Kết hợp dữ liệu liên kết vào trạng thái để đảm bảo tính xác thực.
3. Mã hóa/giải mã (Encryption/Decryption):
4. Dữ liệu gốc được xử lý để tạo ra bản mã hoặc khôi phục lại dữ liệu ban đầu, đồng thời tạo mã xác thực (tag).



Hình 1. 1: Quá trình mã hoá sử dụng thuật toán ASCON



Hình 1. 2: Quá trình giải mã sử dụng thuật toán ASCON

1.2.3. Tính bảo mật và hiệu quả

ASCONE được thiết kế với mục tiêu đơn giản nhưng mạnh mẽ, mang lại các ưu điểm như:

- **Khả năng chống lại các tấn công:** Thuật toán sử dụng số vòng biến đổi linh hoạt (12 vòng trong giai đoạn khởi tạo và 6 vòng trong các bước xử lý tiếp theo) để đảm bảo khả năng chống lại các tấn công phân tích vi sai (differential cryptanalysis) và tuyến tính (linear cryptanalysis).
- **Hiệu quả tính toán:** ASCONE yêu cầu ít tài nguyên phần cứng, phù hợp với các thiết bị có bộ nhớ và tốc độ xử lý hạn chế.
- **Tính linh hoạt:** Hỗ trợ cả mã hóa được xác thực và hàm băm, mở rộng khả năng ứng dụng trong các hệ thống bảo mật khác nhau.

1.3 Mục tiêu và phạm vi báo cáo

a) Mục tiêu

Báo cáo này nhằm:

- Triển khai thuật toán mã hóa và giải mã ASCONE dưới dạng thiết kế VLSI, hiện thực hóa thuật toán dưới mô hình phần cứng.
- Xác minh tính đúng đắn của chức năng mã hóa và giải mã qua mô phỏng và kiểm tra trên thiết kế phần cứng.
- Cung cấp một mô hình tham chiếu cho việc triển khai ASCONE trên các thiết bị tài nguyên hạn chế, làm nền tảng cho các ứng dụng thực tiễn như bảo mật IoT hoặc các hệ thống nhúng.

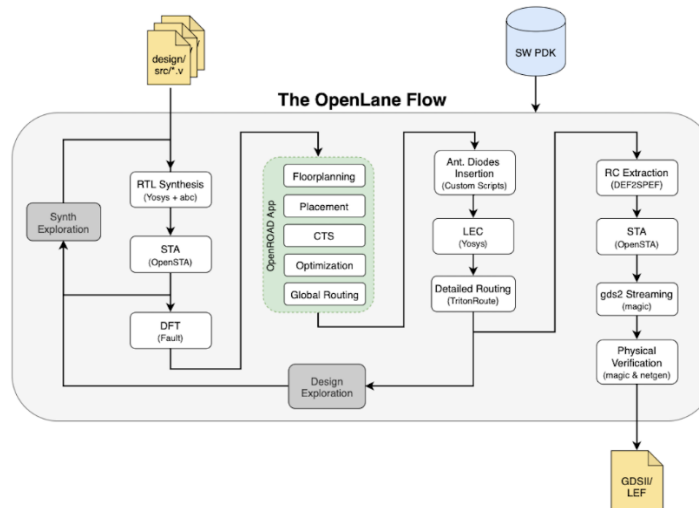
b) Phạm vi

- **Thuật toán ASCONE:** Chỉ tập trung vào chức năng mã hóa và giải mã, bỏ qua các thành phần khác như hàm băm được xác thực.
- **Thiết kế phần cứng:** Triển khai thiết kế trên công nghệ SKY130nm, bao gồm mô hình hóa bằng Verilog và thực hiện Place and Route (PnR) để tạo layout.
- **Kiểm tra và xác minh:** Chỉ thực hiện các bài kiểm tra chức năng và khả năng hoạt động đúng của mạch, không mở rộng đến tối ưu hóa hiệu suất hoặc phân tích tiêu thụ năng lượng.

Phạm vi này đảm bảo tập trung vào việc cung cấp một bản thiết kế cơ bản nhưng đầy đủ, phục vụ cho mục đích nghiên cứu và kiểm chứng.

1.4. Các công cụ thiết kế và phát triển

OpenLane là một khung thiết kế mã nguồn mở dành cho việc thiết kế ASIC (Application-Specific Integrated Circuit), hỗ trợ toàn bộ quy trình từ RTL (Register Transfer Level) đến GDSII (Graphics Data System II). Đây là một giải pháp tự động hóa mạnh mẽ được phát triển trên nền tảng các công cụ mã nguồn mở như Yosys, OpenROAD, Magic, và KLayout, với khả năng tích hợp và tối ưu hóa cao. OpenLane thường được sử dụng kết hợp với PDK (Process Design Kit) như SkyWater 130nm PDK, giúp giảm chi phí và rào cản trong thiết kế vi mạch.



Hình 1. 3: Flow về khung thiết kế mã nguồn mở OpenLane

1.4.1. Ngôn ngữ mô tả phần cứng Verilog

Verilog là một ngôn ngữ mô tả phần cứng (HDL - Hardware Description Language) được sử dụng để thiết kế và mô phỏng các mạch điện tử, đặc biệt là các vi mạch số như FPGA và ASIC. Verilog cho phép mô tả hành vi và cấu trúc của phần cứng ở nhiều cấp độ, từ logic cổng (gate level) đến cấp độ hệ thống (system level). Với cú pháp đơn giản và tương tự ngôn ngữ lập trình C, Verilog đã trở thành một công cụ phổ biến trong lĩnh vực thiết kế vi mạch, hỗ trợ cả thiết kế mô phỏng và tổng hợp để tạo ra các thiết kế phần cứng thực tế.



Hình 1. 4: Ngôn ngữ mô tả phần cứng Verilog

1.4.2. Công cụ tổng hợp logic

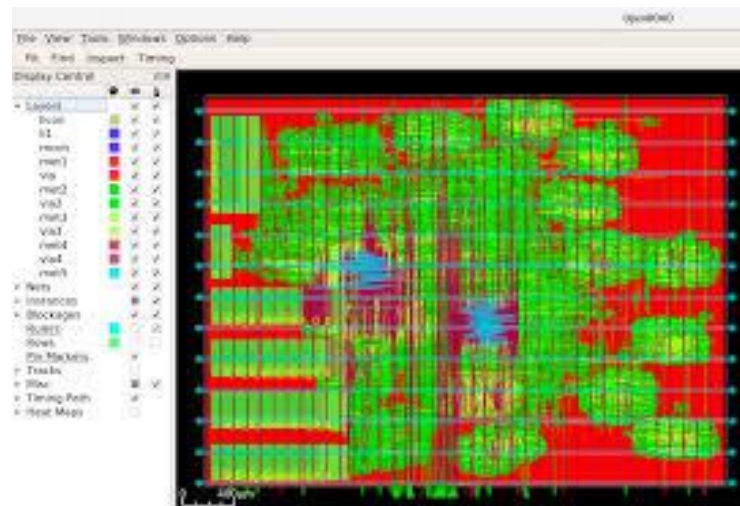
Yosys là một công cụ mã nguồn mở được sử dụng trong bước synthesis (tổng hợp) của quy trình thiết kế vi mạch. Nhiệm vụ chính của Yosys là chuyển đổi mã RTL (thường viết bằng Verilog) thành netlist logic, thể hiện các cổng logic (AND, OR, NOT,...) và kết nối giữa chúng. Ngoài ra, Yosys còn thực hiện các tối ưu hóa logic, giảm thiểu số lượng cổng và ánh xạ netlist vào thư viện chuẩn (standard cell library) phù hợp với công nghệ chế tạo chip. Công cụ này là bước khởi đầu quan trọng để chuẩn bị cho các bước thiết kế tiếp theo trong OpenLane.



Hình 1. 5: Logo của công cụ tổng hợp Yosys

1.4.2 OpenROAD

OpenROAD là một công cụ mã nguồn mở được sử dụng trong bước physical design (thiết kế vật lý) của quy trình thiết kế ASIC. Nó tự động hóa các giai đoạn chính, bao gồm floorplanning (xác định bố cục và vùng đặt các cell), placement (sắp xếp các cell trên chip), clock tree synthesis (CTS) (xây dựng cây đồng hồ để đồng bộ tín hiệu), và routing (định tuyến các kết nối giữa các cell). Ngoài ra, OpenROAD còn hỗ trợ kiểm tra và tối ưu hóa các yếu tố quan trọng như công suất, hiệu năng, và độ trễ, đảm bảo thiết kế tuân thủ các ràng buộc công nghệ và sẵn sàng cho chế tạo.



Hình 1. 6: Hình ảnh các công cụ trong OpenROAD thực hiện ra layout

1.4.3. Công cụ kiểm tra DRC và LVS

Magic là một công cụ mã nguồn mở được sử dụng trong thiết kế vi mạch, đặc biệt trong các bước physical design và signoff. Magic chuyên dụng trong việc thực hiện Design Rule Check (DRC) và Layout vs. Schematic (LVS) check, giúp đảm bảo rằng layout vi mạch tuân thủ các quy tắc thiết kế vật lý và nhất quán với sơ đồ mạch logic. Công cụ này cũng hỗ trợ vẽ layout và xuất các tệp GDSII, chuẩn đầu ra quan trọng cho quá trình chế tạo chip. Với khả năng tích hợp cao và giao diện trực quan, Magic là một công cụ hữu ích trong các dự án thiết kế ASIC và nghiên cứu vi mạch.



Hình 1. 7: Logo của công cụ Magic

1.4.4. Công cụ xem và chỉnh sửa layout

KLayout là một công cụ mã nguồn mở mạnh mẽ, chủ yếu được sử dụng để xem và chỉnh sửa layout của vi mạch trong quá trình thiết kế vật lý. KLayout hỗ trợ các chức năng như visualization, DRC/LVS checking, và layout editing, giúp kỹ sư dễ dàng kiểm tra và điều chỉnh thiết kế của vi mạch trước khi xuất ra tệp GDSII. Công cụ này cung cấp giao diện người dùng trực quan và có khả năng xử lý nhiều định dạng tệp layout, làm cho nó trở thành một công cụ linh hoạt trong quy trình thiết kế vi mạch. KLayout cũng hỗ trợ mở rộng qua các script, cho phép người dùng tự động hóa các quy trình kiểm tra và xử lý layout, từ đó tăng hiệu quả và độ chính xác trong thiết kế.



Hình 1. 8: Logo của công cụ Klayout

Kết Chương 1

Trong chương này, chúng ta đã trình bày bối cảnh và tổng quan về ASCON – một tiêu chuẩn mã hóa tiên tiến được thiết kế để đáp ứng các yêu cầu về bảo mật và hiệu suất trong các ứng dụng bảo mật hiện đại. ASCON không chỉ nổi bật vì cấu trúc đơn giản và dễ triển khai mà còn vì khả năng chịu đựng các cuộc tấn công hiện đại như tấn công thời gian và tấn công cường hóa. Chúng ta cũng đã giới thiệu các công cụ sử dụng trong quy trình thiết kế và đánh giá ASCON, bao gồm các phần mềm mã nguồn mở như Yosys, OpenLane, và OpenROAD, hỗ trợ tự động hóa quy trình từ tổng hợp logic đến thiết kế vật lý. Các công cụ này đóng vai trò quan trọng trong việc tối ưu hóa hiệu suất và bảo mật của ASCON, đồng thời đảm bảo rằng thiết kế cuối cùng tuân thủ các tiêu chuẩn công nghệ và bảo mật cao.

CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC RTL

2.1 Thiết kế các khối chức năng

2.1.1. Thông số kỹ thuật trong ASCON

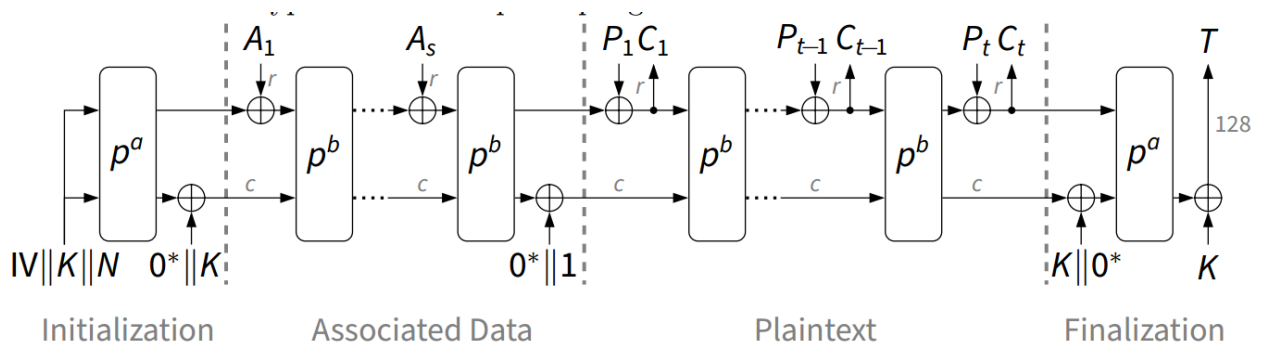
Ascon là một họ mã hóa nhẹ được thiết kế để cung cấp bảo mật mạnh mẽ trong các ứng dụng tài nguyên giới hạn. Nó hỗ trợ mã hóa xác thực (AEAD), nổi bật với thiết kế đơn giản, khả năng chống lại các cuộc tấn công kênh bên và hiệu suất tốt trên cả phần cứng lẫn phần mềm, được tiêu chuẩn hóa bởi NIST cho các hệ thống IoT và nhúng.

Mật mã	Kích thước Bit					Rounds	
	Key	nonce	Tag	Rate	Capacity	p^a	p^b
Ascon-128	128	128	128	64	256	12	6

Bảng 2. 1: Bảng thông số kỹ thuật trong ASCON

2.1.2. Module chính ASCON

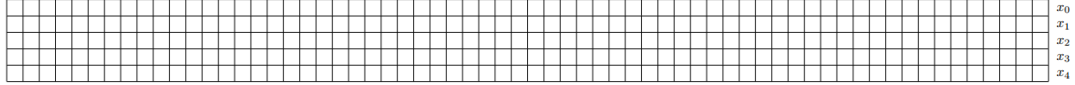
Ascon gồm 4 giai đoạn chính: Khởi tạo (Initialization) để thiết lập trạng thái ban đầu từ khóa và nonce, Xử lý dữ liệu liên kết (Associated Data Processing) để đảm bảo tính toàn vẹn của dữ liệu không mã hóa, Mã hóa/Giải mã (Encryption/Decryption) để xử lý dữ liệu mật, và Kết thúc (Finalization) để tạo tag xác thực bảo vệ tính toàn vẹn và xác thực dữ liệu.



Hình 2. 1: Các giai đoạn hoạt động của ASCON

a) Khởi tạo

Ascon sử dụng một trạng thái nội bộ 320 bit, được chia thành 5 từ 64-bit: x_0, x_1, x_2, x_3, x_4 . Bit ít quan trọng nhất (Least Significant Bit - LSB) nằm ở vị trí cực phải của từ x_4 , trong khi bit quan trọng nhất (Most Significant Bit - MSB) nằm ở vị trí cực trái của từ x_0 . Trạng thái này tuân theo một quy ước chỉ mục hàng ngược của các bit, đảm bảo tính nhất quán khi thực hiện các phép toán trên trạng thái trong quá trình mã hóa.



Bảng 2. 2: $S = x_0 \parallel x_1 \parallel x_2 \parallel x_3 \parallel x_4$

Các vector khởi tạo của Ascon có độ dài 64 bit và được thiết kế dựa trên các tham số quan trọng: kích thước khóa (k), kích thước tốc độ (r), số vòng hoán vị cuối (a) và số vòng hoán vị lỗi (b). Quá trình này bao gồm:

- Điền từng vòng 8 bit: Các giá trị khởi tạo được sắp xếp vào trạng thái nội bộ theo thứ tự ưu tiên.
- Đệm phần còn lại: Các bit chưa được sử dụng trong quá trình điền sẽ được đệm bằng các giá trị cố định để đảm bảo trạng thái ban đầu đầy đủ và sẵn sàng cho các phép toán tiếp theo.

$$IV_{k,r,a,b} \leftarrow k \parallel a \parallel b \parallel 0^{160-k}$$

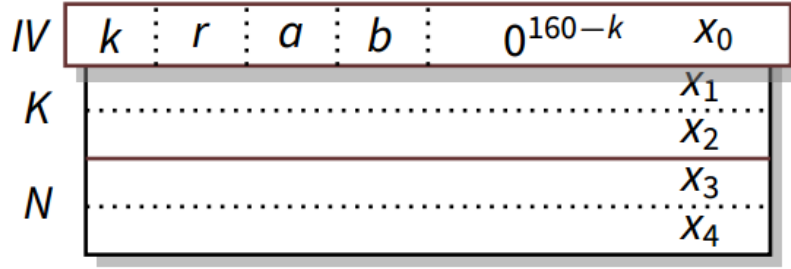
$$\text{ASCON-128} = -80400c0600000000$$

Sau đó, State được điền bằng IV trong x_0 , Key tiếp theo trong x_1, x_2 và Nonce trong phần còn lại x_3, x_4

$$S \leftarrow IV \parallel K \parallel V$$

Sau đó, State được chạy qua hoán vị cuối p^a sau khi được thêm vào bằng một khóa đệm theo cách sau:

$$S \leftarrow p^a(S) \oplus (0^{320-k} \parallel K)$$



Hình 2. 2: Bước khởi tạo

b) Xử lý dữ liệu liên quan (Associated Data Processing)

Sau khi khởi tạo, chúng tôi xử lý Dữ liệu liên kết bằng cách đầu tiên là đệm Dữ liệu liên kết A và chia nó thành các Khối s r -Bit theo cách sau:

$$A_1, \dots, A_s \leftarrow A \parallel 1 \parallel 0^{r-1-(|A| \bmod r)}$$

Sau đó, mỗi Khối dữ liệu liên kết A_i được tiêu hóa bằng cách đưa nó vào hoán vị lồi (permutation)

$$S \leftarrow p^b((S_r \oplus A_i) \parallel S_c), 1 \leq i \leq s$$

Sau khi tất cả các Khối được tiêu hóa, chúng tôi thêm Hằng số Phân tách Miền 1 bit để tăng thêm bảo mật trước các cuộc tấn công hoán đổi Dữ liệu Liên kết và Văn bản thuần túy

$$S \leftarrow S \oplus (0^{319} \parallel 1)$$

c) Mã hoá

Trong mỗi vòng lặp, một khối bản rõ đã được đệm P_i với $i=1, \dots, t_i$ sẽ được XOR với r bit đầu tiên (S_r) của trạng thái nội bộ S , sau đó một khối bản mã C_i được trích xuất. Đối với mỗi khối, trừ khối cuối cùng, toàn bộ trạng thái nội bộ S sẽ được biến đổi thông qua hoán vị p^b bằng b vòng lặp.

$$C_i \leftarrow S_r \oplus P_i$$

$$S \leftarrow \begin{cases} p^b(C_i \| S_c) & \text{if } 1 \leq i < t \\ C_i \| S_c & \text{if } 1 \leq t \end{cases}$$

Khối văn bản mã hóa cuối cùng C_t sau đó được cắt bớt theo độ dài của khối văn bản thuần túy cuối cùng chưa được đệm sao cho độ dài của nó nằm giữa 0 và $r - 1$ bit và tổng độ dài của văn bản mã hóa C hoàn toàn giống với độ dài của văn bản thuần túy gốc P :

$$\tilde{C}_t \leftarrow \lfloor C_t \rfloor_{|P| \bmod r}$$

d) Giải mã

Trong mỗi lần lặp lại ngoại trừ lần lặp cuối cùng, khối văn bản thuần túy P_i được tính bằng cách xor-ing khối văn bản mã hóa C_i với r bit đầu tiên S_r của trạng thái nội bộ. Sau đó, r bit đầu tiên của trạng thái nội bộ, S_r , được thay thế bằng C_i . Cuối cùng, đối với mỗi khối văn bản mã hóa ngoại trừ khối cuối cùng, trạng thái nội bộ được chuyển đổi bằng hoán vị b -round p^b :

$$P_i \leftarrow S_r \oplus C_i$$

$$S \leftarrow p^b(C_i \| S_c), \quad 1 \leq i < t$$

Đối với khối văn bản mã hóa bị cắt bớt cuối cùng C_t với $0 \leq l < r$ bit, quy trình sẽ khác:

$$\tilde{P}_t \leftarrow \lfloor S_r \rfloor_\ell \oplus \tilde{C}_t$$

$$S \leftarrow (S_r \oplus (\tilde{P}_t \| 1 \| 0^{r-1-\ell})) \| S_c$$

e) Kết thúc

Trong quá trình hoàn thiện, khóa bí mật K được xored đến trạng thái nội bộ và trạng thái được chuyển đổi bằng phép hoán vị p^a sử dụng a vòng. Thẻ T bao gồm 128 bit cuối cùng của trạng thái được xored với 128 bit cuối cùng của khóa K :

$$S \leftarrow p^a(S \oplus (0^r \| K \| 0^{c-k}))$$

$$T \leftarrow [S]^{128} \oplus [K]^{128}$$

Thuật toán mã hóa trả về thẻ T cùng với văn bản mã hóa $C_1 \| \dots \| C_t$. Thuật toán giải mã trả về văn bản thuần túy $P_1 \| \dots \| P_t$ chỉ khi giá trị thẻ được tính toán trùng khớp với giá trị thẻ đã nhận.

Authenticated Encryption $\mathcal{E}_{k,r,a,b}(K, N, A, P)$	Verified Decryption $\mathcal{D}_{k,r,a,b}(K, N, A, C, T)$
Input: key $K \in \{0, 1\}^k$, $k \leq 160$, nonce $N \in \{0, 1\}^{128}$, associated data $A \in \{0, 1\}^*$, plaintext $P \in \{0, 1\}^*$ Output: ciphertext $C \in \{0, 1\}^{ P }$, tag $T \in \{0, 1\}^{128}$	Input: key $K \in \{0, 1\}^k$, $k \leq 160$, nonce $N \in \{0, 1\}^{128}$, associated data $A \in \{0, 1\}^*$, ciphertext $C \in \{0, 1\}^*$, tag $T \in \{0, 1\}^{128}$ Output: plaintext $P \in \{0, 1\}^{ C }$ or $\perp$
Initialization $S \leftarrow IV_{k,r,a,b} \ K \ N$ $S \leftarrow p^a(S) \oplus (0^{320-k} \ K)$ Processing Associated Data if $ A > 0$ then $A_1 \dots A_s \leftarrow r$ -bit blocks of $A \ 1 \ 0^*$ for $i = 1, \dots, s$ do $S \leftarrow p^b((S_r \oplus A_i) \ S_c)$ $S \leftarrow S \oplus (0^{319} \ 1)$ Processing Plaintext $P_1 \dots P_t \leftarrow r$ -bit blocks of $P \ 1 \ 0^*$ for $i = 1, \dots, t-1$ do $S_r \leftarrow S_r \oplus P_i$ $C_i \leftarrow S_r$ $S \leftarrow p^b(S)$ $S_r \leftarrow S_r \oplus P_t$ $\tilde{C}_t \leftarrow [S_r]_{ P \bmod r}$ Finalization $S \leftarrow p^a(S \oplus (0^r \ K \ 0^{320-r-k}))$ $T \leftarrow [S]^{128} \oplus [K]^{128}$ return $C_1 \ \dots \ C_{t-1} \ \tilde{C}_t, T$	Initialization $S \leftarrow IV_{k,r,a,b} \ K \ N$ $S \leftarrow p^a(S) \oplus (0^{320-k} \ K)$ Processing Associated Data if $ A > 0$ then $A_1 \dots A_s \leftarrow r$ -bit blocks of $A \ 1 \ 0^*$ for $i = 1, \dots, s$ do $S \leftarrow p^b((S_r \oplus A_i) \ S_c)$ $S \leftarrow S \oplus (0^{319} \ 1)$ Processing Ciphertext $C_1 \dots C_{t-1} \tilde{C}_t \leftarrow r$ -bit blocks of C , $0 \leq \tilde{C}_t < r$ for $i = 1, \dots, t-1$ do $P_i \leftarrow S_r \oplus C_i$ $S \leftarrow C_i \ S_c$ $S \leftarrow p^b(S)$ $\tilde{P}_t \leftarrow [S_r]_{ \tilde{C}_t } \oplus \tilde{C}_t$ $S_r \leftarrow S_r \oplus (\tilde{P}_t \ 1 \ 0^*)$ Finalization $S \leftarrow p^a(S \oplus (0^r \ K \ 0^{320-r-k}))$ $T^* \leftarrow [S]^{128} \oplus [K]^{128}$ if $T = T^*$ return $P_1 \ \dots \ P_{t-1} \ \tilde{P}_t$ else return $\perp$

Hình 2. 3: Thuật toán mã hoá và giải mã của ASCON

2.1.3. Các module phụ trong ASCON

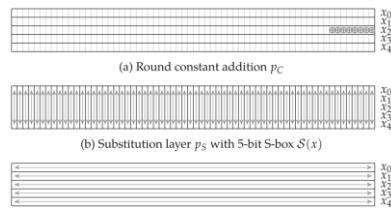
Module Permutation trong thuật toán ASCON là thành phần cốt lõi, đảm nhận vai trò tạo ra tính ngẫu nhiên và khuếch tán thông tin trong các bước mã hóa và giải mã. Đây là một mạng hoán vị dựa trên cấu trúc SPN (Substitution-Permutation Network), kết hợp giữa các bước thay thế (substitution) và hoán vị (permutation) để đảm bảo tính phi tuyến tính và khả năng chống lại các tấn công mật mã học, như tấn công phân tích vi sai (Differential Cryptanalysis) và phân tích tuyến tính (Linear Cryptanalysis). Module này được tối ưu hóa để thực hiện hiệu quả các phép toán XOR, AND, và các phép dịch vòng, giúp giảm thiểu độ phức tạp phần cứng và năng lượng tiêu thụ. Trong thiết kế VLSI, module Permutation được triển khai sao cho đạt hiệu suất cao, đáp ứng yêu cầu về tốc độ và diện tích, đồng thời duy trì tính bảo mật mạnh mẽ.

A) Permutation

Các thành phần chính của các lược đồ Ascon là hai hoán vị 320 bit p^a và p^b . Các hoán vị này lặp đi lặp lại áp dụng một phép biến đổi vòng dựa trên p vòng SPN , lần lượt bao gồm ba bước p_C, p_S, p_L :

$$p = p_L \circ p_S \circ p_C.$$

p^a và p^b chỉ khác nhau ở số vòng. Số vòng a và số vòng b là các tham số bảo mật có thể điều chỉnh. Đối với mô tả và ứng dụng của các phép biến đổi vòng, trạng thái 320 bit S được chia thành năm thanh ghi 64 bit words x_i , $S = x_0 \parallel x_1 \parallel x_2 \parallel x_3 \parallel x_4$



Hình 2. 4: Đăng ký các từ của trạng thái 320 bit S và hoạt động của module Permutation

B) Round constraint

Bước cộng hằng số p^C thêm hằng số vòng c_r vào thanh ghi từ x_2 của trạng thái S trong vòng i . Cả hai chỉ số r và i đều bắt đầu từ số không và chúng ta sử dụng $r = i$ cho p^a và $r = i + a - b$ cho p^b

$$x_2 \leftarrow x_2 \oplus c_r.$$

p^{12}	p^8	p^6	Constant c_r	p^{12}	p^8	p^6	Constant c_r
0			00000000000000f0	6	2	0	0000000000000096
1			00000000000000e1	7	3	1	0000000000000087
2			00000000000000d2	8	4	2	0000000000000078
3			00000000000000c3	9	5	3	0000000000000069
4	0		00000000000000b4	10	6	4	000000000000005a
5	1		00000000000000a5	11	7	5	000000000000004b

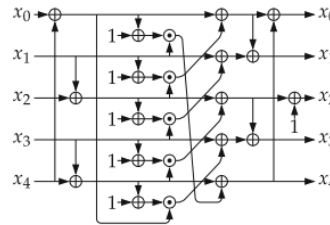
Bảng 2. 3: Bảng round constraint c_r được sử dụng mỗi vòng

C) Substitution Layer

Lớp thay thế p_s cập nhật trạng thái S với 64 ứng dụng song song của hộp S 5 bit $S(x)$ được định nghĩa trong Hình 4a cho mỗi bit-slice của năm thanh ghi $x_0 \dots x_4$. Nó thường được triển khai ở dạng bit-slice này với các hoạt động được thực hiện trên toàn bộ các từ 64 bit.

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e	1f
$S(x)$	4	b	1f	14	1a	15	9	2	1b	5	8	12	1d	3	6	1c	1e	13	7	e	0	d	11	18	10	c	1	19	16	a	f	17

Bảng 2. 4: Bảng tra cứu hộp S 5 bit $S(x)$



Hình 2. 5: Kiến trúc module Substitution Layer

D) Linear Diffusion Layer

Lớp khuếch tán tuyến tính p_L cung cấp sự khuếch tán trong mỗi thanh ghi 64 bit của x_i . Nó áp dụng một hàm tuyến tính $i(x_i)$ được định nghĩa trong hình sau:

$$x_i \leftarrow \Sigma_i(x_i), \quad 0 \leq i \leq 4.$$

$$x_0 \leftarrow \Sigma_0(x_0) = x_0 \oplus (x_0 \ggg 19) \oplus (x_0 \ggg 28)$$

$$x_1 \leftarrow \Sigma_1(x_1) = x_1 \oplus (x_1 \ggg 61) \oplus (x_1 \ggg 39)$$

$$x_2 \leftarrow \Sigma_2(x_2) = x_2 \oplus (x_2 \ggg 1) \oplus (x_2 \ggg 6)$$

$$x_3 \leftarrow \Sigma_3(x_3) = x_3 \oplus (x_3 \ggg 10) \oplus (x_3 \ggg 17)$$

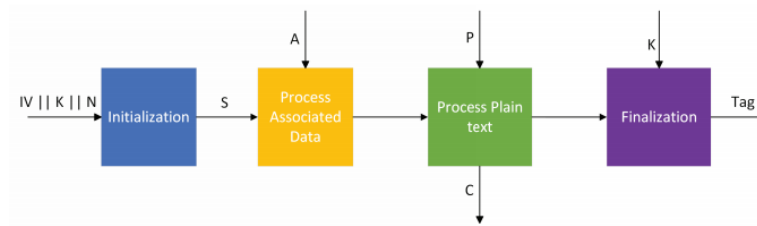
$$x_4 \leftarrow \Sigma_4(x_4) = x_4 \oplus (x_4 \ggg 7) \oplus (x_4 \ggg 41)$$

Hình 2. 6: Chức năng của module Linear Diffusion Layer

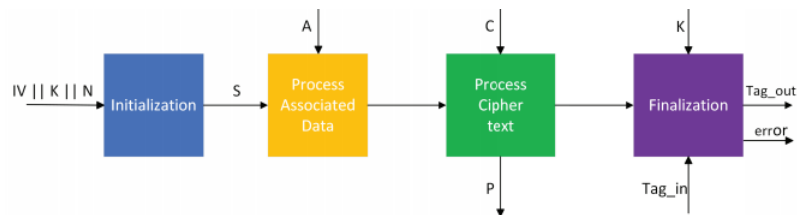
2.2. Thiết kế RTL cho ASCON

2.2.1 Sơ đồ logic

Sơ đồ logic trong thiết kế VLSI cho ASCON thuật toán là một bản mô tả trực quan về các khối chức năng và cách chúng kết nối với nhau.



Hình 2. 7: Block diagram của ASCON mã hoá



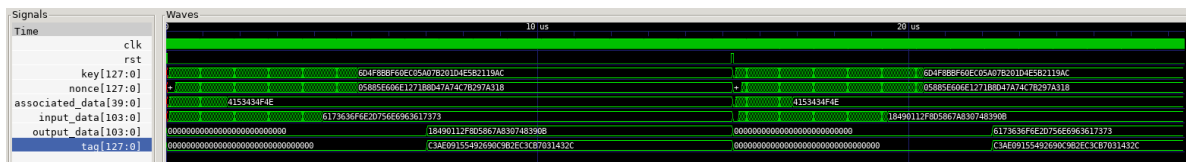
Hình 2. 8: Block diagram của ASCON giải mã

2.2.2. Testbench

Một testbench cho ASCON được sử dụng để kiểm tra và xác minh hoạt động của module thiết kế ASCON, đảm bảo rằng nó tuân thủ đúng chức năng mã hóa, giải mã và tạo mã xác thực (authentication).

```
Start!
Start encryption!
Key: 36a7c5dfb07602d03d900ea72d908cd6
Nonce: 02c42f30370938dc6a3d3a63d94bd18c
AD: 4153434f4e
PT: 6173636f6e2d756e6963617373
Encryption Done! It took 47 clock cycles
CT: 18490112f8d5867a830748390b
Tag: c3ae09155492690c9b2ec3cb7031432c
Start decryption!
Key: 6d4f8bbf60ec05a07b201d4e5b2119ac
Nonce: 05885e606e1271b8d47a74c7b297a318
AD: 4153434f4e
CT: 18490112f8d5867a830748390b
Decryption Done! It took 47 clock cycles
PT: 6173636f6e2d756e6963617373
Tag: c3ae09155492690c9b2ec3cb7031432c
```

Hình 2. 9: Quá trình xác minh đúng về chức năng của ASCON



Hình 2. 10: Waveform của ASCON

Kết chương 2

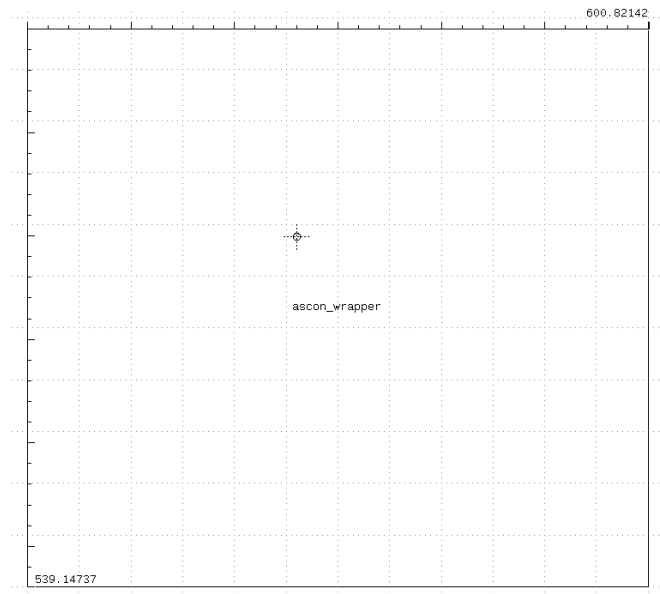
Sau khi đã giới thiệu các module chính và phụ của Ascon, cùng với việc phân tích chi tiết block diagram của từng khối, chúng ta đã có một cái nhìn toàn diện về cấu trúc và chức năng của hệ thống. Việc trình bày testbench cho Ascon giúp đảm bảo rằng các module hoạt động đúng như thiết kế, cung cấp cơ sở để kiểm chứng hiệu suất và độ tin cậy của thuật toán mã hóa này. Qua đó, Ascon không chỉ được xây dựng hoàn chỉnh mà còn sẵn sàng cho các ứng dụng thực tế.

CHƯƠNG 3. HOÀN THIỆN LAYOUT VÀ KẾT QUẢ

3.1 Quá trình Floorplaning

Trong bước floorplanning, kích thước bố cục được xác định là 600 x 540. Đây là bước quan trọng để sắp xếp hợp lý các ô chuẩn trên mặt phẳng hai chiều, đảm bảo tối ưu không gian và phân bố hợp lý các vùng chức năng, đồng thời hỗ trợ kết nối hiệu quả giữa các khối logic trong thiết kế ASIC.

Với kích thước 600 x 540, việc floorplanning không chỉ đảm bảo đủ không gian cho tất cả các ô chuẩn mà còn tính đến các yếu tố như vùng phân phối điện, mạng xung nhịp và khoảng cách giữa các khối logic quan trọng. Cách bố trí này giúp giảm thiểu độ trễ tín hiệu, tối ưu hóa hiệu suất và đảm bảo thiết kế đáp ứng các yêu cầu về thời gian và điện năng.



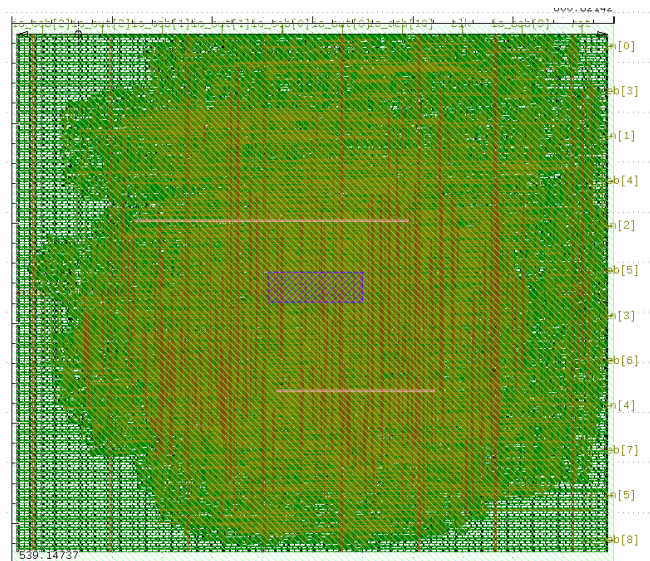
Hình 3. 1: Quá trình tạo khung để chứa các Standard Cell

3.2 Quá trình Place và Route

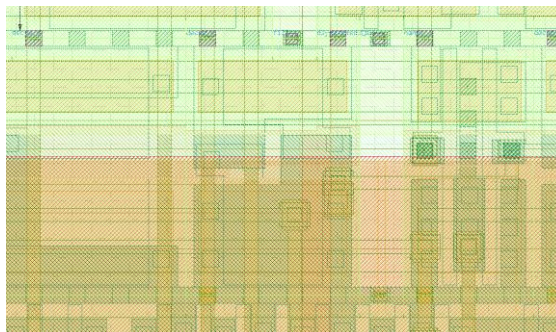
Place and Route là giai đoạn quan trọng trong thiết kế ASIC, chuyển đổi từ netlist logic thành bố cục vật lý. Trong bước placement, các ô chuẩn được sắp xếp trên mặt phẳng

hai chiều sao cho tối ưu không gian và khoảng cách kết nối, đồng thời vẫn đáp ứng các yêu cầu về hiệu năng và thời gian. Giai đoạn này cũng bao gồm việc đặt các khối chức năng lớn, đảm bảo chúng được bố trí hợp lý theo các kết nối và phụ thuộc logic.

Sau khi placement hoàn tất, bước routing sẽ vẽ các kết nối kim loại giữa các ô chuẩn và các khối chức năng. Routing bao gồm việc tạo mạng lưới phân phối điện, kết nối dây xung nhịp đến các thanh ghi, và đảm bảo không có hiện tượng giao cắt hoặc xung đột. Quá trình này tối ưu hóa các đường dây dẫn, giảm độ trễ tín hiệu và đảm bảo các thông số thiết kế như công suất và hiệu năng. Kết quả cuối cùng là một bố cục vật lý hoàn chỉnh, sẵn sàng cho các bước kiểm tra và chế tạo.



Hình 3. 2: Quá trình đặt các cell vào bên trong khung

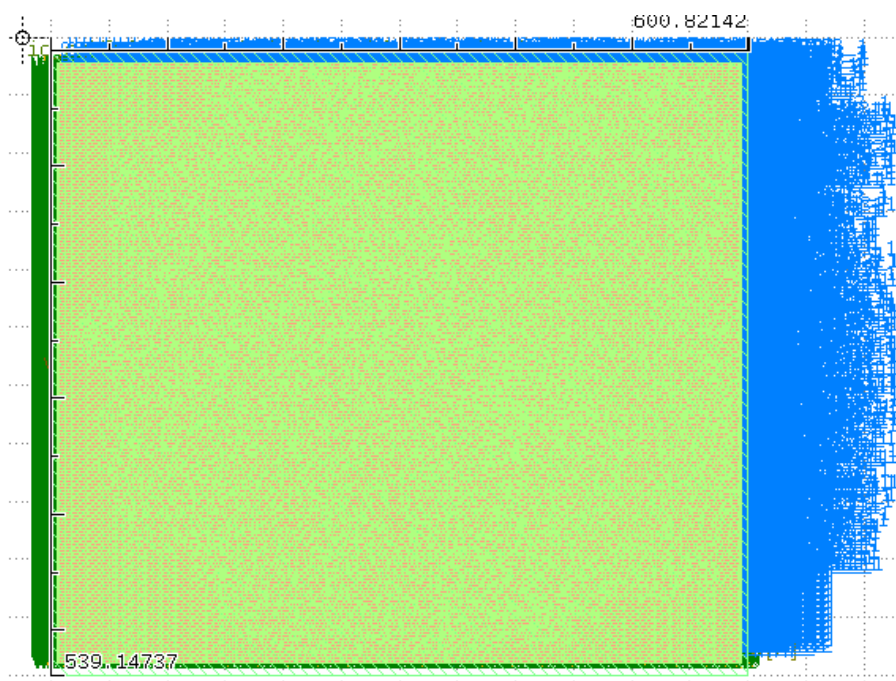


Hình 3. 3: Quá định định tuyến của các cell với nhau

3.3 Quá trình hoàn thiện layout

Quá trình chuyển đổi từ ngôn ngữ mô tả phần cứng (HDL) sang bố cục ASIC bao gồm nhiều bước thử nghiệm và tổng hợp để đảm bảo thiết kế hoạt động đúng ý định. Đầu tiên, mã HDL được tổng hợp thành netlist, sử dụng thư viện các ô chuẩn – các khối logic cơ bản của mạch tích hợp (IC). Netlist mô tả cách kết nối các ô chuẩn và dây dẫn.

Sau đó, phân tích thời gian tĩnh (STA) xác minh tín hiệu đáp ứng thời gian xung nhịp. Bố cục được tạo bằng cách đặt các ô trên mặt phẳng hai chiều và vẽ kết nối kim loại, đồng thời xây dựng mạng phân phối điện và dây xung nhịp. Mô phỏng cấp cổng (GL) kiểm tra chức năng và STA GL đảm bảo không vi phạm thời gian do các thuộc tính vật lý như độ dài dây. Cuối cùng, mô phỏng công suất GL cung cấp ước tính tiêu thụ điện năng chính xác hơn, xác nhận tính khả thi của thiết kế sau tổng hợp và bố cục.



Hình 3. 4: Quá trình hoàn thiện layout của ASCON

3.4 Đặc tả của thiết kế và phân tích

Công cụ thiết kế bố cục của OpenLane cung cấp khả năng phân tích và trực quan hóa các module trong thiết kế, làm nổi bật số lượng cell và diện tích tương ứng của chúng. Trong thiết kế này, Khối lõi Ascon và các module con bao gồm 16080 cell chuẩn.

Module name	Cell Count	Diện tích (μm^2)	Năng lượng tiêu thụ (W)
Ascon_wrapper	16080	128567	1.01e-02

Bảng 3. 1: Bảng tổng hợp thông số cần thiết kế layout hoàn thành

Kết Chương 3

Sau khi hoàn thành quá trình bố trí và định tuyến, thiết kế đã được tối ưu hóa toàn diện về vị trí các ô chuẩn, kết nối dây dẫn và mạng lưới phân phối điện. Kết quả là một bố cục vật lý hoàn chỉnh, đảm bảo đáp ứng các yêu cầu về hiệu năng, thời gian và tiêu thụ điện năng, đồng thời sẵn sàng cho các bước kiểm tra và xuất file GDSII để chế tạo chip.

KẾT LUẬN

Sau khi hoàn thành toàn bộ quy trình thiết kế từ RTL (Register Transfer Level) đến GDSII, chúng ta đã đạt được một kết quả thiết kế hoàn chỉnh và sẵn sàng cho quá trình chế tạo chip. Bắt đầu từ việc xác minh chức năng của mã HDL, thiết kế đã được chuyển đổi thành netlist thông qua bước tổng hợp, đảm bảo logic tuân thủ các ràng buộc về thời gian và công suất. Tiếp theo, quá trình floorplanning và place and route được thực hiện để bố trí các ô chuẩn và kết nối dây dẫn, tối ưu hóa không gian và đảm bảo hiệu năng. Trong giai đoạn này, mạng phân phối điện và xung nhịp cũng được xây dựng, đồng thời các phân tích thời gian tĩnh (STA) và kiểm tra điện năng đã xác nhận rằng thiết kế đáp ứng các yêu cầu kỹ thuật.

Mô phỏng cấp công (GL) và các bước kiểm tra bổ sung đã đảm bảo rằng thiết kế duy trì đầy đủ chức năng sau khi được chuyển đổi thành dạng vật lý, tính đến các yếu tố vật lý như độ dài dây và đặc điểm công nghệ chế tạo. Cuối cùng, file GDSII được tạo ra chứa tất cả thông tin cần thiết để chế tạo, bao gồm bố trí các khối logic, kết nối dây dẫn và các lớp kim loại. Thành công của quy trình này không chỉ khẳng định tính khả thi của thuật toán mã hóa Ascon mà còn tạo ra một thiết kế hiệu quả, sẵn sàng ứng dụng vào thực tế. Quy trình từ RTL đến GDSII đã chứng minh khả năng triển khai thuật toán này trên phần cứng, mở ra tiềm năng cho các ứng dụng mã hóa nhẹ trong IoT và các thiết bị nhúng.

Hướng phát triển trong tương lai sẽ tập trung vào việc tối ưu hóa hơn nữa thiết kế của Ascon để cải thiện hiệu năng, giảm tiêu thụ điện năng và tối ưu diện tích chip. Một số hướng nghiên cứu cụ thể bao gồm:

- Tối ưu hóa công suất và hiệu suất: Thực hiện các phân tích và cải tiến nhằm giảm tiêu thụ điện năng trong các ứng dụng thực tế, đặc biệt là các thiết bị IoT có yêu cầu cao về tiết kiệm năng lượng.
- Nâng cao khả năng bảo mật: Tích hợp thêm các biện pháp chống tấn công vật lý (physical attacks) như tấn công kênh bên (side-channel attacks) và phân tích lỗi (fault injection analysis) vào thiết kế để tăng cường độ an toàn.

- Nghiên cứu trên các công nghệ chế tạo mới: Thử nghiệm thiết kế trên các công nghệ bán dẫn tiên tiến như 7nm hoặc 5nm, giúp giảm kích thước chip và tăng hiệu suất tổng thể.

Các hướng phát triển này không chỉ mở rộng phạm vi ứng dụng của Ascon mà còn nâng cao giá trị của nó như một giải pháp mã hóa nhẹ hiệu quả và an toàn trong các hệ thống hiện đại.

TÀI LIỆU THAM KHẢO

- [1] Kandi, A., Baksi, A., Gerlich, T., Guilley, S., Gan, P., Breier, J., ... & Bhasin, S. (2023). Hardware Implementation of ASCON. In *NIST LWC Workshop*.
- [2] Dobraunig, C., Eichlseder, M., Mendel, F., & Schl  ffer, M. (2021). Ascon v1. 2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34, 1-42.
- [3] Zheng, Y., Luo, Q., Li, Q., & Lv, Y. (2024). Quantum circuit implementations of lightweight authenticated encryption ASCON. *The Journal of Supercomputing*, 1-16.
- [4] Athanasiou, G. S., Boufeas, D., & Konstantopoulou, E. (2024, March). A Robust ASCON Cryptographic Coprocessor for Secure IoT Applications. In *2024 Panhellenic Conference on Electronics & Telecommunications (PACET)* (pp. 1-6). IEEE.